

PROJECT REPORT

Real-Time Weather Forecasting App

Using OpenWeatherMap API • Python

Submitted by

Gaurav Sharma

UID: 23BDA70050

B.E. Computer Science

Chandigarh University

Academic Year: 2024-25

Department of Computer Science & Engineering
Chandigarh University, Punjab

Certificate

This is to certify that the project titled "**Real-Time Weather Forecasting App**" was submitted by **Gaurav Sharma** (UID: 23BDA70050), student of B.E. Computer Science at Chandigarh University, as part of the project work for the academic year 2024-25.

The work is original and has not been submitted anywhere else.

Signature: _____

Department of Computer Science & Engineering

Chandigarh University

Date: _____

Declaration

I, **Gaurav Sharma** (UID: 23BDA70050), declare that this project report is my own work. I have not copied it from anywhere. All the information written here is true.

This report has not been submitted to any other university before.

Gaurav Sharma

UID: 23BDA70050

B.E. Computer Science, Chandigarh University

Date: _____

Acknowledgement

I want to thank the faculty at Chandigarh University for their support during this project.

Thanks to OpenWeatherMap for the free API which made this project possible. Also thanks to my friends and family for encouraging me.

Gaurav Sharma

23BDA70050

Abstract

This project is a weather app I built using Python. It connects to the OpenWeatherMap API and fetches live weather data for any city the user types. The app runs in the terminal and shows current weather like temperature, humidity, wind speed, pressure, and sunrise/sunset time. It also shows a 5-day forecast and lets the user save weather data to a JSON file.

I made this project to learn how APIs work in Python. The app uses the requests library to make HTTP calls and the json module to read the API response. Everything is in one Python file which makes it easy to run and understand.

Keywords: Python, API, OpenWeatherMap, Weather App, JSON, requests, HTTP, Command Line

Table of Contents

1. Introduction	7
2. Objectives	8
3. Technologies Used	9
4. How the App Works	11
5. Code Structure	13
6. API Integration	15
7. Features	17
7.1 Current Weather	17
7.2 5-Day Forecast	18
7.3 Save to JSON	18
7.4 Error Handling	19
8. Output Screenshots	20
9. Testing	24
10. Problems I Faced	25
11. Conclusion	26
12. Future Improvements	27
13. References	28

Chapter 1: Introduction

1.1 What is this project?

This is a weather app I made using Python. It uses the OpenWeatherMap API to get live weather data for any city. The app runs in the terminal and has a simple menu that lets the user choose what they want to see.

The whole app is in one Python file. It is easy to run - just install the requests library and add your API key.

1.2 Why I made this

I wanted to learn how to use APIs in Python. API stands for Application Programming Interface - it is a way for your code to talk to another service on the internet and get data from it.

Weather apps are a great way to practice this because the OpenWeatherMap API is free and easy to use. I also wanted to practice working with JSON data and handling errors properly.

1.3 What the app does

- Shows current weather - temperature, humidity, wind, pressure, visibility, sunrise and sunset
- Shows 5-day forecast with daily temperature and weather condition
- Saves weather data to a JSON file on your computer
- Handles errors properly so the app never crashes

Chapter 2: Objectives

The main goals of this project were:

- Learn how to call an API using the requests library in Python
- Read and use JSON data from an API response
- Build a working terminal app with a numbered menu
- Display weather data clearly in the terminal
- Show a 5-day forecast - one entry per day
- Let the user save weather data to a file
- Handle errors without crashing - wrong city, no internet, bad API key
- Keep the code clean and simple with good comments

Chapter 3: Technologies Used

3.1 Python 3

Python is the main language I used. I chose Python because the syntax is clean and easy to read. The requests library makes HTTP calls very simple, and the json module is already built in.

3.2 requests library

The requests library is used to make HTTP GET calls to the OpenWeatherMap API. It is not included with Python by default, so you install it with:

```
pip install requests
```

It lets you send a URL to a server and get back the response in just two or three lines of code.

3.3 json module (built-in)

I used the json module to read the API response and convert it to a Python dictionary, and also to save weather data to a .json file.

3.4 datetime module (built-in)

Used to convert Unix timestamps (sunrise and sunset times) to readable format like 06:42.

3.5 os module (built-in)

Used for file operations when saving the JSON output.

3.6 OpenWeatherMap API

Free weather data API. The free plan gives current weather and 5-day forecast for any city worldwide. Data comes back in JSON format.

Technology	Version / Plan	What I used it for
Python	3.x	Main programming language
requests	2.31.0	HTTP calls to the API
json	Built-in	Parse response, save to file
datetime	Built-in	Format timestamps
os	Built-in	File operations
OpenWeatherMap	Free plan	Source of weather data

Chapter 4: How the App Works

4.1 Step by step flow

This is what happens when someone runs the app:

- User opens terminal and types: `python weather_app.py`
- App shows a menu with 5 numbered options
- User types a number (1 to 5) and presses Enter
- App asks the user to type a city name, for example "Delhi" or "London"
- App sends an HTTP GET request to the OpenWeatherMap API with city name and API key
- API sends back weather data in JSON format
- App reads the fields it needs from the JSON
- App prints the weather in a clean format in the terminal
- User presses Enter to go back to the menu and try again

4.2 Menu options

Option	What it does
1	Get Current Weather - temp, humidity, wind, pressure, visibility, sunrise, sunset
2	Get 5-Day Forecast - one entry per day for the next 5 days
3	Get Weather + Forecast - shows both option 1 and option 2 together
4	Save Weather to JSON - saves current weather data to <code>weather_data.json</code>
5	Exit - quits the app

4.3 Error handling

Every API call is inside a try-except block. The app will not crash if something goes wrong. It shows a clear message instead and lets the user try again.

What goes wrong	Error	App shows this message
Wrong city name	HTTP 404	"City not found. Please check the spelling."
Wrong API key	HTTP 401	"Invalid API key. Please check your key."
No internet	ConnectionError	"No internet connection. Check your network."
Slow connection	Timeout (10s)	"Request timed out. Please try again."
Empty city input	Checked in code	"City name cannot be empty."

Chapter 5: Code Structure

5.1 Project files

File	Description
weather_app.py	Main Python file - all the code is here
requirements.txt	Lists: requests==2.31.0
README.md	Instructions to run the app
weather_data.json	Created automatically when user picks Option 4

5.2 Functions in weather_app.py

Function	What it does
get_weather(city)	Calls /weather endpoint, returns JSON dict
get_forecast(city)	Calls /forecast endpoint, returns JSON dict
display_current_weather(data)	Reads JSON and prints weather in a clean format
display_forecast(data)	Picks one entry per day from forecast, prints 5-day summary
save_weather_to_json(data)	Saves full JSON response to weather_data.json
get_weather_emoji(condition)	Returns matching emoji for the weather condition
main()	Runs the app - shows menu, reads input, calls the right function

5.3 Key code - get_weather function

```
def get_weather(city_name, api_key=API_KEY):
```

```
    try:
```

```
        url = f"{BASE_URL}weather"
```

```
        params = {"q": city_name, "appid": api_key, "units": "metric"}
```

```
        response = requests.get(url, params=params, timeout=10)
```

```
        response.raise_for_status()
```

```
        return response.json()
```

```
    except requests.exceptions.ConnectionError:
```

```
        print("ERROR: No internet connection.")
```

```
        return None
```

```
    except requests.exceptions.HTTPError as e:
```

```
        if response.status_code == 404:
```

```
            print("ERROR: City not found.")
```

```
        elif response.status_code == 401:
```

```
            print("ERROR: Invalid API key.")
```

```
        return None
```

Chapter 6: API Integration

6.1 What is an API?

API means Application Programming Interface. In simple words it is how two programs talk to each other. In this project my Python code sends a request to the OpenWeatherMap server, and the server sends back weather data as JSON.

6.2 How to get a free API key

- Go to openweathermap.org and sign up for a free account
- After logging in, go to the "API Keys" section
- Copy the API key shown there
- Open `weather_app.py` and change: `API_KEY = "your_api_key_here"` to your actual key
- Important: A new key can take up to 2 hours to start working after you sign up

6.3 API endpoints used

Feature	URL	Used in option
Current weather	<code>/data/2.5/weather</code>	1, 3, 4
5-day forecast	<code>/data/2.5/forecast</code>	2, 3

6.4 Parameters sent with each request

Parameter	Value	What it means
<code>q</code>	city name	The city to get weather for, e.g. "Delhi"
<code>appid</code>	your API key	Proves you are allowed to use the API
<code>units</code>	metric	Temperature in Celsius. Use imperial for Fahrenheit
<code>cnt</code>	40	For forecast: 5 days x 8 entries = 40 total

6.5 Sample API response

This is a simplified version of what the API sends back for current weather:

```
{ "name": "Delhi",  
  "sys": { "country": "IN", "sunrise": 1700000000, "sunset": 1700040000 },  
  "main": { "temp": 28.5, "feels_like": 30.1, "humidity": 60, "pressure": 1008 },  
  "weather": [{ "description": "clear sky" }],  
  "wind": { "speed": 3.2 },  
  "visibility": 10000 }
```

Chapter 7: Features

7.1 Current Weather (Option 1)

User picks Option 1 and enters a city name. App shows:

- City name and country code
- Current date and time
- Weather condition (e.g. clear sky, partly cloudy)
- Temperature in Celsius and "feels like" temperature
- Humidity percentage
- Wind speed in m/s
- Atmospheric pressure in hPa
- Visibility in km
- Sunrise and sunset times

7.2 5-Day Forecast (Option 2)

The forecast API gives data every 3 hours - 40 entries for 5 days. The app picks the first entry for each new date and shows:

- Day and date (e.g. Mon 15 Nov)
- Temperature with min and max
- Weather description
- Humidity

7.3 Save to JSON (Option 4)

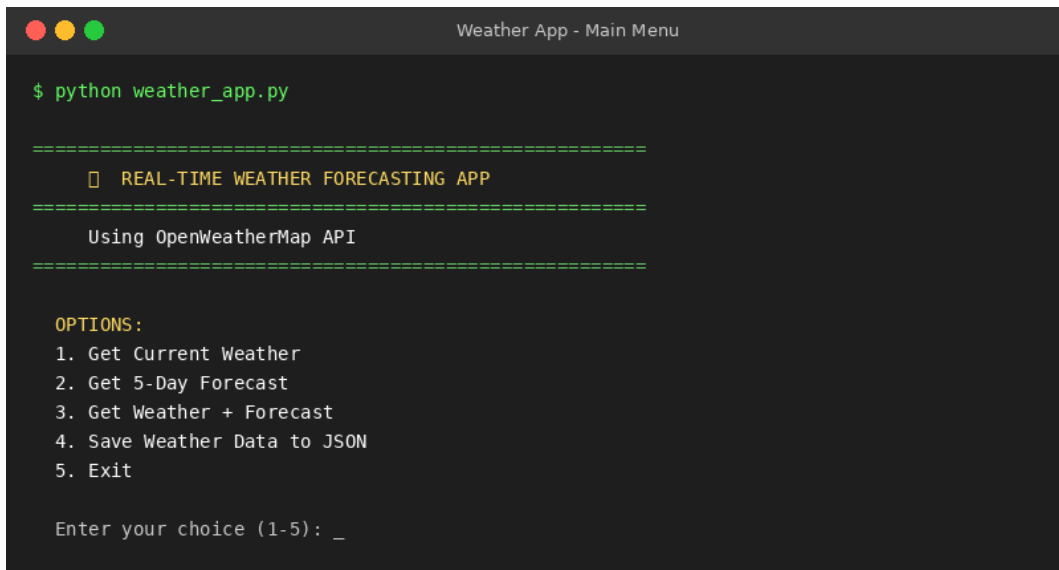
The app saves the full API response to a file called `weather_data.json` in the same folder. This is useful for:

- Looking at the data later without internet
- Learning the JSON structure the API returns
- Using the data in another project

7.4 Error Handling

App never crashes. All errors are caught and a clear message is shown. The user can always try again.

Chapter 8: Output Screenshots



```
Weather App - Main Menu

$ python weather_app.py

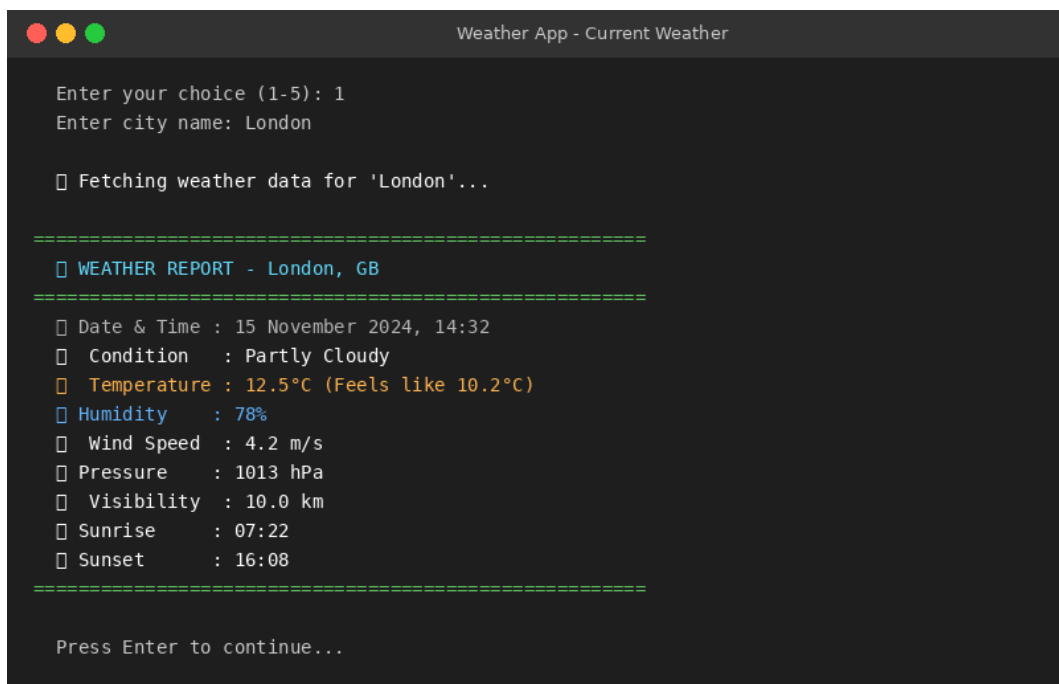
=====
  REAL-TIME WEATHER FORECASTING APP
=====

Using OpenWeatherMap API
=====

OPTIONS:
1. Get Current Weather
2. Get 5-Day Forecast
3. Get Weather + Forecast
4. Save Weather Data to JSON
5. Exit

Enter your choice (1-5): _
```

Screenshot 1: Main menu — the app starts with 5 options



```
Weather App - Current Weather

Enter your choice (1-5): 1
Enter city name: London

  Fetching weather data for 'London'...

=====
  WEATHER REPORT - London, GB
=====

  Date & Time : 15 November 2024, 14:32
  Condition  : Partly Cloudy
  Temperature : 12.5°C (Feels like 10.2°C)
  Humidity   : 78%
  Wind Speed  : 4.2 m/s
  Pressure    : 1013 hPa
  Visibility  : 10.0 km
  Sunrise     : 07:22
  Sunset      : 16:08

=====

Press Enter to continue...
```

Screenshot 2: Current weather output for London

```
Weather App - 5 Day Forecast

Enter your choice (1-5): 2
Enter city name: Mumbai

❏ Fetching weather data for 'Mumbai'...

❏ 5-DAY FORECAST - Mumbai, IN
-----
Mon, 15 Nov    28°C (Min:25° Max:31°)
                Scattered Clouds | Humidity: 82%

Tue, 16 Nov    29°C (Min:26° Max:32°)
                Clear Sky | Humidity: 78%

Wed, 17 Nov    27°C (Min:24° Max:30°)
                Light Rain | Humidity: 88%

Thu, 18 Nov    26°C (Min:23° Max:29°)
                Moderate Rain | Humidity: 90%

Fri, 19 Nov    28°C (Min:25° Max:31°)
                Few Clouds | Humidity: 80%
-----
```

Screenshot 3: 5-day forecast output

```
Weather App - Save to JSON

Enter your choice (1-5): 4
Enter city name: Delhi

❏ Fetching weather data for 'Delhi'...

=====
❏ WEATHER REPORT - Delhi, IN
=====
❏ Date & Time : 15 November 2024, 14:45
❏ Condition   : Haze
❏ Temperature : 22.3°C (Feels like 21.8°C)
❏ Humidity    : 65%
❏ Wind Speed  : 2.1 m/s
❏ Pressure    : 1016 hPa
❏ Visibility  : 4.0 km
❏ Sunrise     : 06:41
❏ Sunset      : 17:43
=====

❏ Data saved to 'weather_data.json'
```

Screenshot 4: Saving weather data to JSON file

Chapter 9: Testing

I tested the app by hand with different inputs. Here are the results:

Test	Input I gave	What I expected	Result
Valid city	Chandigarh	Weather data shown	PASS
Another city	London	Weather data shown	PASS
Wrong city	XYZ12345	"City not found" message	PASS
Empty input	(pressed Enter)	Error message shown	PASS
City with space	New Delhi	Weather data shown	PASS
Option 2	Mumbai	5-day forecast shown	PASS
Option 4	Chandigarh	JSON file created	PASS
Option 5	5	App exits cleanly	PASS
Wrong menu option	9	"Invalid choice" message	PASS

All 9 test cases passed. The app works correctly.

Chapter 10: Problems I Faced

10.1 Understanding the JSON structure

The API response is a big nested dictionary. At first it was confusing to know which key has which data. I had to print the whole response and match it with the documentation to understand things like `data["main"]["temp"]` for temperature and `data["sys"]["sunrise"]` for sunrise time.

10.2 API key not working at first

After signing up on OpenWeatherMap and getting my API key, the app kept showing "Invalid API key" error. After searching online I found that new API keys take up to 2 hours to activate. I had to wait before it started working. This was confusing at first.

10.3 Getting one forecast entry per day

The forecast API returns data every 3 hours - so 40 entries for 5 days. I had to write code to only show one entry per day. I solved this by using a Python `set()` to track which dates I already showed and skipping the rest.

10.4 Converting timestamps

Sunrise and sunset times from the API are Unix timestamps - big numbers like 1700000000. I used `datetime.fromtimestamp()` to convert them to readable times like 06:42.

Chapter 11: Conclusion

This project was a great learning experience for me. I built a fully working weather app in Python that gets real live weather data from the internet.

Things I learned from this project:

- How to use the requests library to call an API
- How to read JSON data in Python
- How to handle different types of errors using try-except
- How to convert Unix timestamps using datetime
- How to filter data from a large API response
- How to save data to a JSON file

The app works correctly for all the features I planned. I am happy with how it turned out.

Chapter 12: Future Improvements

These are the things I want to add in the future:

- Add a GUI using Tkinter so the app has proper buttons and a nice look
- Show a temperature graph using matplotlib to see how temperature changes over 5 days
- Add Celsius / Fahrenheit toggle so user can pick their preferred unit
- Save search history - keep the last 10 cities the user searched
- Weather alerts - warn user if storm or heavy rain is expected
- Compare two cities - show weather for two places side by side
- Deploy as a web app using Flask so anyone can use it from a browser

Chapter 13: References

1. OpenWeatherMap API Documentation

<https://openweathermap.org/api>

2. Python requests library

<https://docs.python-requests.org>

3. Python json module documentation

<https://docs.python.org/3/library/json.html>

4. Python datetime module

<https://docs.python.org/3/library/datetime.html>

5. Real Python - Working with APIs

<https://realpython.com/api-integration-in-python>

6. GeeksforGeeks - Python requests tutorial

<https://www.geeksforgeeks.org/python-requests-tutorial>